

Dart & Git — Homework Assignment

Sessions 1–4 · Intermediate Level · Duration: 2 Days

Instructions: Complete all tasks in a single Dart file (`main.dart`) unless stated otherwise. Push your final work to a GitHub repository named `dart-homework`. Each section is worth the points shown. Total: **20 points**.

Before You Start

Before writing a single line of Dart, you need to set up your workspace. Create a folder called `dart-homework`, initialize a Git repository inside it, create your `main.dart` file, and push everything to a GitHub repository with the same name. This is your `main` branch.

The Rule for Every Task

Every task you work on gets its own **branch**, named after its part — for example `task-1` for all of Part 1, `task-2` for Part 2, and so on. All your code for that part lives there.

Each time you finish a sub-task (1.1, 1.2, 1.3...), you make a **commit** for it with a short message describing what you did. That way, every sub-task is a checkpoint inside its branch.

Once you finish all sub-tasks in a part and you're happy with everything, you switch back to `main` and **merge your branch into it**. Only then do you move on and create the next branch for the next part.

By the end of the assignment, your `main` branch should contain all completed work, your repository history should show **4 branches** — one per part — each carrying its own commits, all merged cleanly into `main`.

Part 1 — Variables, Data Types & Null Safety

4 pts

Task 1.1 — Profile Card

(1 pt)

Declare the following variables using the most appropriate keyword (`var`, `final`, or `const`):

- The app name: `"StudentTracker"` — this never changes at all
- A student's full name — assigned once at runtime, never changes after
- A student's current GPA — can change over time
- A student's scholarship amount — optional (may or may not exist)

Print a formatted profile card using string interpolation:

```
App: StudentTracker
Student: [name]
GPA: [gpa]
Scholarship: [amount or "None"]
```

Use ?? for the scholarship fallback.

Task 1.2 — Type Conversion

(1 pt)

Given the following strings:

```
String rawScore = "87";  
String rawAverage = "14.75";
```

1. Convert `rawScore` to `int`, add 13, and print the result
2. Convert `rawAverage` to `double`, multiply by 2, and print the result
3. Convert the doubled average back to a `String` and print its character length

Task 1.3 — Null Safety in Practice

(1 pt)

```
String? city;  
String? email;
```

1. Print the length of `city` safely
2. Assign `"walid@dart.dz"` to `email`
3. Print `email` length using `?.`
4. Declare a `late String username` — assign it the value `"admin"` before printing it

Task 1.4 — Spot & Fix

(1 pt)

The code below has **3 errors**. Find them, comment the error line then explain each one in a comment, and write the corrected version.

```
void main() {  
  const double taxRate = 0.19;  
  taxRate = 0.20;  
  
  int price = 49.99;  
  
  String? label;  
  print("Label length: " + label.length);  
}
```

Part 2 — Control Flow & Operators

4 pts

Task 2.1 — Grade Classifier

(1 pt)

Write a program that takes an `int score` (0–100) and prints the grade using `if / else if / else`:

Score	Grade
90–100	A -- Excellent
75–89	B -- Good
60–74	C -- Average
Below 60	F -- Failed

Then rewrite **only the final else** using a ternary operator to print either "Pass" or "Fail" on a second line.

Task 2.2 — Access Control

(1 pt)

Declare three variables:

```
bool isLoggedIn = true;
bool isAdmin    = false;
int age         = 20;
```

Write conditions using `&&`, `||`, and `!` to print:

1. "Welcome back!" if the user is logged in **and** is at least 18
2. "Admin panel unlocked" if the user is an admin **or** age is over 25
3. "Guest mode" if the user is **not** logged in

Task 2.3 — Day Planner

(1 pt)

Use a `switch` statement on a `String` `day` variable. Print:

- "Work from office" for "Monday", "Tuesday", "Wednesday"
- "Work from home" for "Thursday", "Friday"
- "Rest day" for "Saturday", "Sunday"
- "Invalid day" for anything else

Hint: Multiple `case` labels can fall through to the same body in Dart — look up how.

Task 2.4 — Loop Patterns

(1 pt)

1. Use a `for` loop to print all **odd numbers** from 1 to 20 (use `continue` to skip evens)
2. Use a `while` loop to count down from 10 to 1, then print "Lift off!"
3. Use a `do-while` loop that asks (simulates) a PIN entry — start with `int attempts = 0`, keep looping while the pin is wrong, and stop after **3 attempts** even if never correct. Print "Access granted" or "Account locked" accordingly (hardcode the correct PIN as 1234 and a test PIN as 9999)

Part 3 — Functions

4 pts

Task 3.1 — Pure Functions

(1 pt)

Write the following three functions:

1. `int clamp(int value, int min, int max)` — returns `value` if it's between `min` and `max`, `min` if below, `max` if above
2. `String repeatString(String s, int times)` — returns `s` repeated `times` times (e.g. `repeatString("ab", 3) → "ababab"`)
3. `bool isPrime(int n)` — returns `true` if `n` is a prime number

Call each function in `main()` with at least two test cases and print the results.

Task 3.2 — Named & Optional Parameters

(1 pt)

Write a function `String buildProfile` with:

- A **required named** parameter `String name`
- A **required named** parameter `int age`
- An **optional named** parameter `String role` with a default value of `"Student"`

The function returns a formatted string:

```
[name] | Age: [age] | Role: [role]
```

Call it three times: once with all three parameters, once with only `name` and `age`, and once with a different role like `"Instructor"`.

Task 3.3 — Arrow Functions

(1 pt)

Rewrite each of these as **arrow functions**:

```
double celsiusToFahrenheit(double c) {  
    return c * 9 / 5 + 32;  
}  
  
bool isLongString(String s) {  
    return s.length > 10;  
}  
  
int square(int n) {  
    return n * n;  
}
```

Call each one and print the result.

Task 3.4 — Function + Loop Combination

(1 pt)

Write a function `List<int> getFibonacci(int count)` that returns the first `count` numbers of the Fibonacci sequence as a list.

- Use a `for` loop inside the function
- Call it with `count = 10` and print the result
- Then call it again with `count = 5` and print the **sum** of that list using another loop

Part 4 — Collections

5 pts

Task 4.1 — List Operations

(1 pt)

Start with:

```
List<String> courses = ["Dart", "Flutter", "Git", "Python", "SQL"];
```

1. Print the third course by index
2. Add "Firebase" at the end
3. Insert "OOP" at index 1
4. Remove "Python"
5. Print all courses with their 1-based position: 1. Dart, 2. OOP, ...
6. Use `.where()` to print only courses with more than 3 characters

Task 4.2 — List `.map()` & Chaining

(1 pt)

Given:

```
List<int> scores = [55, 78, 42, 90, 63, 81, 38, 74];
```

1. Use `.where()` to keep only scores of 60 or above
2. Chain `.map()` to convert each passing score to a string: "Pass: 78"
3. Print the result as a list
4. Also compute and print the **average** of all original scores using a loop

Task 4.3 — Map: Grade Book

(1.5 pts)

Create `Map<String, Map<String, dynamic>` `students` with at least **5 students**. Each student maps to:

```
{
  'score': int,
  'attendance': int,    // percentage 0-100
  'passed': bool
}
```

1. Set 'passed' to true if `score >= 60 && attendance >= 75`, otherwise false
2. Iterate using `for-in` on `entries` and print each student's full record
3. Use `.values` to compute the class score average

4. Build a `List<String>` of students who failed and print it
5. Add a new student after the map is declared and print the updated length

Task 4.4 — Set Operations

(1.5 pts)

```
Set<String> backendSkills = {"Dart", "SQL", "Firebase", "REST",  
    "Git"};  
Set<String> frontendSkills = {"Dart", "Flutter", "HTML", "CSS",  
    "Git"};
```

1. Print skills required in **both** tracks
2. Print skills **only** in backend
3. Print skills **only** in frontend
4. Print **all unique skills** across both tracks
5. Add "Dart" to `backendSkills` again — confirm the size did not change
6. Convert the union set to a `List`, sort it alphabetically, and print it

Part 5 — Quiz

3 pts

Answer each question directly in a comment inside `main.dart`. Each correct answer is worth **0.5 pts**.

Q1 (0.5 pt) — True / False

A `const` variable in Dart can be assigned a value that is only known at runtime.

True **False**

Q2 (0.5 pt) — True / False

A `Set` in Dart automatically removes duplicate values and does not guarantee insertion order.

True **False**

Q3 (0.5 pt) — True / False

The following code compiles and runs without error:

```
void main() {  
    late String name;  
    print(name);  
}
```

True **False**

Q4 (0.5 pt) — Multiple Choice

What does the following code print?

```
void main() {  
    List<int> numbers = [1, 2, 3, 4, 5];  
    var result = numbers.where((n) => n % 2 == 0).map((n) => n *  
        10);  
    print(result);  
}
```

- A) [2, 4]
- B) [20, 40]
- C) (20, 40)
- D) [10, 20, 30, 40, 50]

Q5 (0.5 pt) — Multiple Choice

Which of the following is the correct way to declare a function with a required named parameter in Dart?

- A) void greet(String name) {}
- B) void greet({String name}) {}
- C) void greet({required String name}) {}
- D) void greet([String name]) {}

Q6 (0.5 pt) — Multiple Choice

What is the output of the following code?

```
void main() {  
    String? city;  
    print(city?.length ?? 0);  
}
```

- A) null
- B) 0
- C) A runtime error
- D) ""

*Designed by **HAMMOUTI Walid***